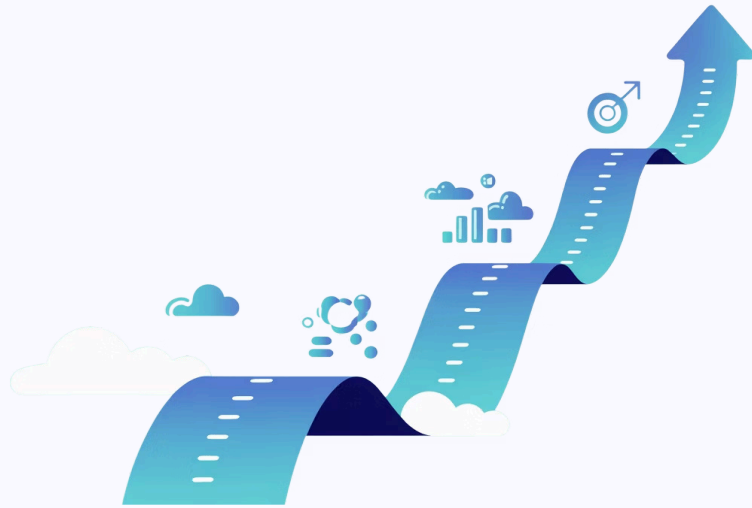




MLOps Roadmap

Your Beginner-Friendly Guide to Machine Learning Operations

by [TechWorld with Nana](#)



**TECHWORLD
WITH NANA**

Provided by TechWorld with Nana

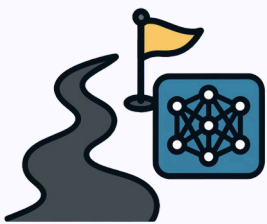
I'm Nana, Co-Founder of TechWorld with Nana.

As a Cloud and DevOps engineer, I'm dedicated to helping engineers build the most valuable and highly-demanded DevOps and Cloud skills.

Through my [YouTube channel](#) and my comprehensive [DevOps bootcamps](#), I've helped 1,000,000s of engineers master the tools and concepts that drive modern software development.



About This MLOps Roadmap



MLOps Roadmap

This detailed guide provides a complete step-by-step roadmap to understanding and learning MLOps.

Whether you're just starting out or looking to level up your existing skills, this document gives you a structured guide to master MLOps.

Happy learning!

Nana Janashia

Co-Founder TechWorld with Nana

Table of Contents

1	What is MLOps and Why Do We Need It?
2	The Real-World Problem MLOps Solves
3	What Does an MLOps Workflow Look Like?
4	Your MLOps Learning Roadmap
5	Who Needs to Learn MLOps?
6	Next Steps in Your Journey

What is MLOps and Why We Need It?

The Kitchen Analogy:

Think of MLOps like a professional kitchen. Even the best chef (data scientist) can't serve food to hundreds of customers (users) without a proper kitchen setup. You need organized stations, quality equipment, food safety standards, and a system to handle orders smoothly.

That's exactly what MLOps does for machine learning!

-  **MLOps stands for Machine Learning Operations.** It's the practice of getting machine learning models from a data scientist's laptop into real products where actual users can benefit from them - and keeping those models running smoothly over time.

Why Traditional Software Development Isn't Enough

Here's the thing: building machine learning systems is different from building regular software.

Traditional software is like building a house with a detailed blueprint. Once built, it works as designed unless you change the code.

But ML systems are more like building a house that continuously rearranges itself based on weather, the people living in it, and changes in the neighborhood.

Machine learning models have unique challenges:



They depend heavily on data, which constantly changes in the real world



Their performance can get worse over time (we call this "**model drift**")



They need regular retraining with new data



Involves many specialists - data engineers, data scientists, ML engineers, and ops teams

The Real Impact

Companies like Netflix use ML models to recommend content to millions of users. If they built a model once and never maintained it, the recommendations would become terrible over time as new shows are added and people's tastes change.

Instead, they need systems to continuously collect viewing data, retrain their models, and deploy updated versions. That's MLOps in action!

The Real-World Problem MLOps Solves

Let me tell you a story that happens all the time in companies...

The Banking Fraud Detection Story



Imagine a bank building a fraud detection system to automatically flag suspicious transactions and protect customers.

The data science team builds an amazing model. In their development environment - running Python notebooks with clean historical data - the model correctly identifies 95% of fraudulent transactions while keeping false alarms low.

Everyone is excited! They deploy it to production servers to start using it with real customer transactions.

But then things start breaking down:

1

The Language Mismatch

The data scientists built their model using Python libraries on their laptops.

But the bank's production servers run Java applications (many banks prefer Java for its reliability and security in transaction-heavy systems).

The engineering team has to rewrite everything in Java, and when they finally deploy it, the model takes 3 seconds to evaluate each transaction - way too slow when the bank processes thousands of transactions per minute.

2

The Environment Nightmare

It's not just the programming language.

The model was developed on laptops but deployed on cloud-based Kubernetes clusters.

Suddenly there are mismatches in system libraries, dependency versions, memory, and CPU.

Things that worked perfectly in development start throwing errors or running incredibly slowly in production.

The Real-World Problem MLOps Solves

3

The Data Drift Problem

After a month in production, the model starts missing many fraud cases.

Why? New types of fraud have emerged that weren't in the original training data.

The model hasn't learned about these new patterns, so it can't catch them.

4

The Reproducibility Issue

When they try to update the model with new fraud examples, they can't get the same good results as before.

Nobody documented exactly how the original model was built - what data was used, what settings were chosen, what steps were taken to clean the data.

5

The Blind Spot Problem

The team only discovered the model was missing fraud cases after customers complained.

There was no system actively monitoring whether the model was still working correctly.

What They Actually Needed

To fix these problems, the team needed:

- ✓ **Consistent working environments** - The model should run the same way on a laptop as on production servers
- ✓ **Performance testing** - Test if the model is fast enough before putting it in front of real customers
- ✓ **Data monitoring** - Constantly check if incoming data still looks like the training data
- ✓ **Complete documentation** - Save every detail of how the model was built so it can be recreated
- ✓ **Real-time monitoring** - A dashboard showing how well the model performs at all times
- ✓ **Easy updates** - A way to update the model quickly when needed, without disrupting the service

That's where MLOps comes in!

What Does an MLOps Workflow Look Like?

Let's continue with our banking fraud detection example and see how MLOps solves each problem.

Solving Problem 1: Deployment with Docker & Kubernetes



With MLOps, we use containers (Docker!) - think of them as sealed packages that contain the model and everything it needs to run.

The data scientists package their Python model in a Docker container. This container can run anywhere - on their laptops or on the bank's servers - and will work exactly the same way.

But we go further! For production, we deploy these containers on Kubernetes clusters. Kubernetes handles scaling the fraud detection service up during busy shopping periods and down during quiet times.

The best part? We can define this entire setup as code using tools like Terraform. This means the entire pipeline is documented, version-controlled, and reproducible.

Solving Problem 2: Performance Testing with CI/CD



MLOps includes automated testing through CI/CD (Continuous Integration/Continuous Deployment) pipelines.

Before the model goes live, the system runs tests to check if it's fast enough. If the banking app's CI/CD pipeline shows the model is too slow, the team can optimize it or use techniques like batch processing to speed it up.

Tools like Jenkins or GitLab CI run these tests automatically whenever someone wants to deploy a new model version.

Solving Problem 3: Data Drift Monitoring

MLOps constantly watches incoming data using validation tools like TensorFlow Data Validation or Great Expectations.

These tools compare new data with what the model was trained on. If customers start behaving differently - say, suddenly making lots of large transfers overseas late at night when that used to be rare - the monitoring tools notice this shift quickly.

They can alert the team or even automatically trigger model retraining with examples of the new fraud types.

What Does an MLOps Workflow Look Like?

Solving Problem 4: Experiment Tracking

With MLOps, everything is recorded automatically using tools like MLflow or DVC (Data Version Control):

- The exact data used for training
- All steps used to clean and prepare data
- All model settings and parameters
- The results of each training attempt

For our banking model, this means the team can see exactly how the original model was built and recreate it perfectly, now with new fraud examples added.

Solving Problem 5: Real-Time Monitoring



MLOps sets up dashboards using tools like Prometheus and Grafana that show:

- How accurate the model is right now
- How many transactions it's processing
- Whether it's flagging too many legitimate transactions
- If it's missing actual fraud

The banking team sees these metrics in real-time and gets instant alerts if the fraud detection rate drops below an acceptable level.

Solving Problem 6: Automated Updates

When new fraud patterns emerge, the monitoring system alerts the team. Some advanced systems can even automatically kick off retraining jobs.

The model learns to catch new fraud tricks quickly, and deployment tools like ArgoCD can automatically replace the old production model with the better new one - with zero downtime.

What Does an MLOps Workflow Look Like?

Here's what a full MLOps workflow looks like for our fraud detection system:

The Complete End-to-End Workflow



This entire process can happen automatically, ensuring the bank always has the best fraud detection model running!



Your MLOps Learning Roadmap

Now let's break down exactly what you need to learn and in what order. This roadmap will take you from complete beginner to confident MLOps practitioner.

Phase 1: Foundation Skills

Before diving into MLOps directly, you need a solid foundation. Don't skip this phase!



1) Learn Python Programming

Python is the language of machine learning and MLOps.

What to focus on:

- Basic syntax and data structures (lists, dictionaries, sets)
- Functions and object-oriented programming
- Working with files and APIs
- Key libraries: Pandas (data manipulation), NumPy (numerical operations)



Why it matters:

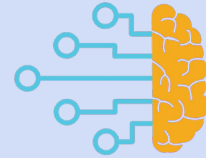
Almost every MLOps tool and framework uses Python. You'll write scripts to automate workflows, process data, and integrate different systems.

2) Understand Machine Learning Basics

You don't need to be a data science expert, but you should understand what ML models do and how they work.

What to focus on:

- What is machine learning?
- How models are trained and evaluated
- Common algorithms (linear regression, decision trees, neural networks)
- Model evaluation metrics (accuracy, precision, recall, F1 score)
- Popular ML frameworks: scikit-learn, TensorFlow, or PyTorch basics



Why it matters:

You can't deploy and maintain ML models effectively if you don't understand how they work. You need to know what data scientists are doing so you can build the right infrastructure.

3) Learn Command Line Basics

Get comfortable working with the terminal and bash scripting.

What to focus on:

- Navigating directories and managing files
- Basic bash scripting for automation
- Working with SSH to access remote servers
- Understanding Linux/Unix basics



Why it matters:

Most MLOps work happens in cloud environments accessed through command line. Plus, automation scripts often use bash.

4) Understand Git and Version Control

Learn to track changes and collaborate on code.

What to focus on:

- Git basics: commit, push, pull, branch, merge
- Working with GitHub or GitLab
- Pull requests and code reviews
- Branching strategies

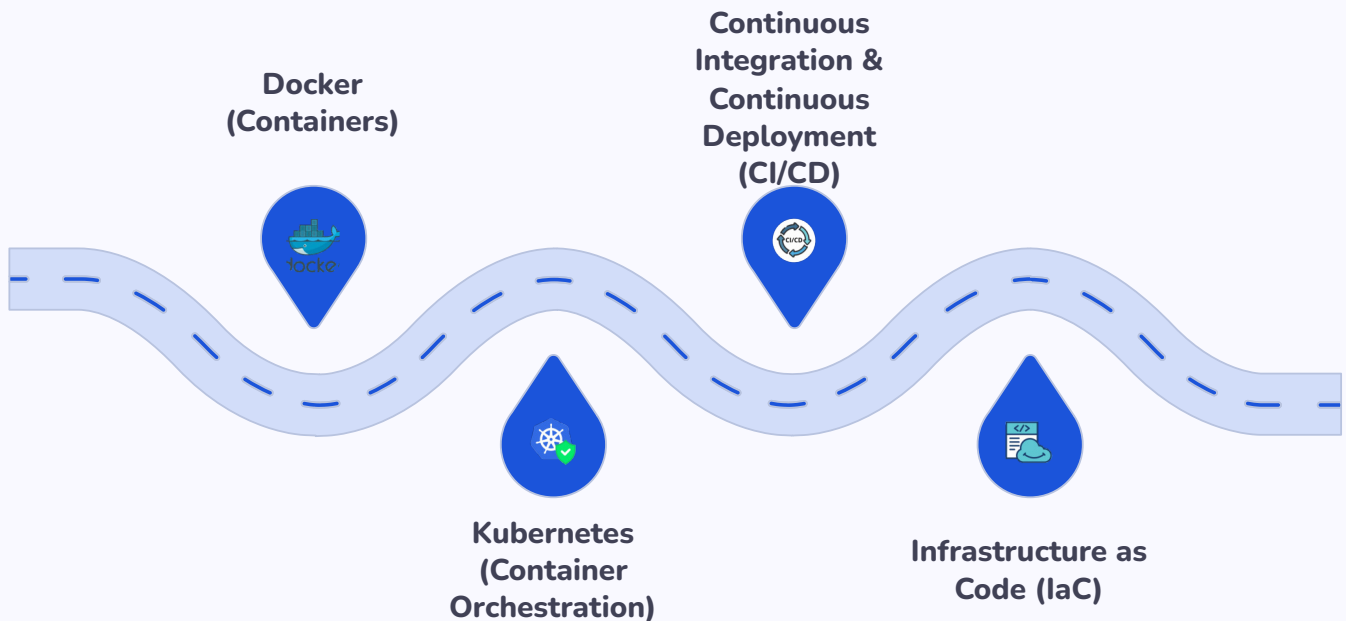


Why it matters:

In MLOps, you version control not just code but also model configurations, training scripts, and infrastructure definitions.

Phase 2: Core DevOps Skills

MLOps builds heavily on DevOps practices. These skills are essential.



1) Master Docker Containers

Containers solve the "it works on my machine" problem.

What to focus on:

- Understanding what containers are and why they matter
- Writing Dockerfiles
- Building and running containers
- Docker Compose for multi-container applications
- Best practices for container images



Why it matters:

Packaging ML models in containers ensures they run consistently across different environments - from data scientist laptops to production servers.



Practical project: Package a simple ML model (like a text classifier) in a Docker container and run it locally.

2) Learn Kubernetes

Kubernetes orchestrates containers at scale.

What to focus on:

- Kubernetes architecture (pods, services, deployments)
- Deploying applications to Kubernetes
- Scaling and updating applications
- Basic kubectl commands
- Understanding how Kubernetes manages workloads



Why it matters:

Production ML systems often run on Kubernetes because it handles scaling, self-healing, and resource management automatically.

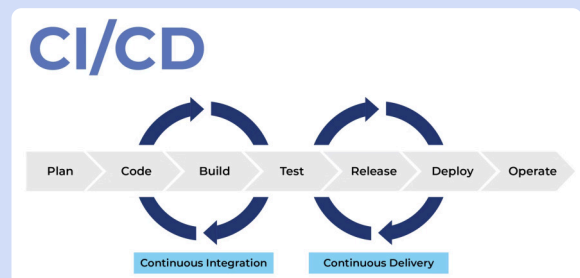
📌 **Practical project:** Deploy your containerized ML model to a local Kubernetes cluster (like Minikube) or a cloud-managed Kubernetes service.

3) Understand CI/CD Pipelines

Automation is the heart of MLOps.

What to focus on:

- What CI/CD means and why it's important
- Popular tools: Jenkins, GitLab CI, GitHub Actions
- Building a simple pipeline
- Automated testing concepts
- Deployment strategies (blue-green, canary)



Why it matters:

CI/CD pipelines automate the entire process from code changes to production deployment, including model training, testing, and deployment.

📌 **Practical project:** Create a CI/CD pipeline that automatically tests and deploys your model when you push code changes.

Phase 3: Cloud Computing Essentials

Most ML systems run in the cloud. Pick one cloud provider to start (AWS, Azure, or Google Cloud).

1) Learn Core Cloud Services

What to focus on:

- Compute services (EC2, Azure VMs, or Google Compute Engine)
- Storage services (S3, Azure Blob Storage, or Google Cloud Storage)
- Managed Kubernetes services (EKS, AKS, or GKE)
- Networking basics (VPCs, security groups)
- IAM (Identity and Access Management)



Why it matters:

You'll deploy ML models on cloud infrastructure, and understanding cloud services helps you design cost-effective, scalable solutions.

📌 **Practical project:** Deploy your ML model on a cloud-managed Kubernetes cluster and expose it via an API endpoint.

2) Learn Infrastructure as Code

Now that you know how cloud resources work, learn to automate creating them.

What to focus on:

- Understanding Infrastructure as Code (IaC) concepts
- Terraform is the most popular IaC tool
- Creating and managing cloud resources programmatically
- Version controlling infrastructure definitions



Why it matters:

IaC lets you create reproducible environments and manage complex infrastructure efficiently. You can spin up entire ML platforms with a single command.

📌 **Practical project:** Write Terraform code to automatically create all the cloud resources you manually created in the previous step (Kubernetes cluster, storage buckets, networking).

Phase 4: MLOps-Specific Tools and Practices

Now we get into the specialized MLOps tools and workflows.

1) Experiment Tracking and Model Registry

Keep track of model training experiments and versions.

What to focus on:

- MLflow for experiment tracking and model registry
- Tracking metrics, parameters, and artifacts
- Comparing different model versions
- Registering and versioning production models



Why it matters:

Data scientists run hundreds of experiments. You need to track what worked, what didn't, and be able to reproduce successful results.

📌 **Practical project:** Use MLflow to track multiple training runs of a model, then register the best version.

2) Data Versioning

Version control for datasets, not just code.

What to focus on:

- DVC (Data Version Control)
- Versioning large datasets efficiently
- Linking data versions to model versions
- Sharing datasets across teams



Why it matters:

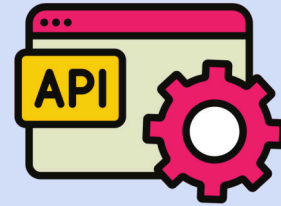
Models are only as good as their training data. When data changes, you need to track those changes and understand how they affect model performance.

3) Model Serving and APIs

Make your models accessible to applications.

What to focus on:

- Building REST APIs with Flask or FastAPI
- Model serving frameworks (TensorFlow Serving, TorchServe)
- API best practices (authentication, rate limiting, error handling)
- Load balancing for high traffic



Why it matters:

Applications need a way to send data to your model and get predictions back. APIs provide that interface.

📌 **Practical project:** Build a REST API that serves predictions from your model and can handle multiple requests per second.

4) Monitoring and Observability

Watch your models in production to catch problems early.

What to focus on:

- Prometheus for metrics collection
- Grafana for dashboards and visualization
- Logging best practices
- Alerting strategies
- Key ML metrics to monitor (accuracy, latency, data drift)



Why it matters:

Models can degrade over time. Monitoring helps you catch issues before they impact users.

📌 **Practical project:** Set up a monitoring dashboard for your deployed model showing prediction latency, throughput, and basic error rates.

5) Feature Stores

Centralized storage for ML features.

What to focus on:

- What feature stores are and why they matter
- Popular tools (Feast, Tecton)
- Feature engineering workflows
- Sharing features across teams



Why it matters:

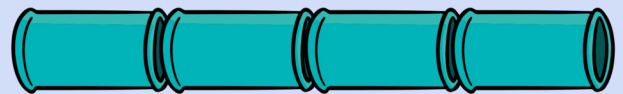
Feature stores solve the problem of feature consistency between training and production, and enable feature reuse across projects.

6) ML Pipelines and Orchestration

Automate the entire ML workflow from data to deployment.

What to focus on:

- Workflow orchestration tools (Apache Airflow, Kubeflow Pipelines)
- Building end-to-end ML pipelines
- Scheduling and triggering workflows
- Pipeline best practices



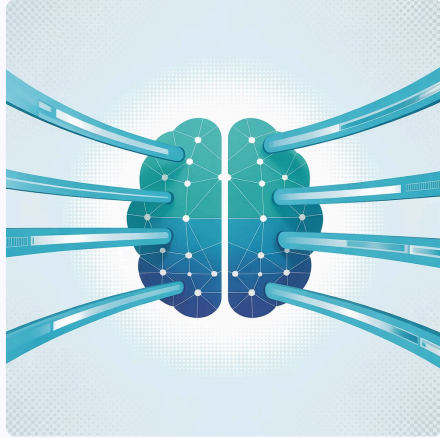
Why it matters:

Manual model training and deployment doesn't scale. Pipelines automate everything from data processing to model deployment.

📌 **Practical project:** Build a complete ML pipeline that automatically trains a model when new data arrives, evaluates it, and deploys it if it performs better than the current production model.

Phase 5: Advanced Topics (Ongoing)

Once you've mastered the basics, these advanced topics will make you stand out.



Model Monitoring and Drift Detection

Go beyond basic monitoring to detect when models need retraining.

What to focus on:

- Detecting data drift and concept drift
- Tools like Evidently AI or Fiddler
- Automated retraining triggers
- A/B testing for model versions

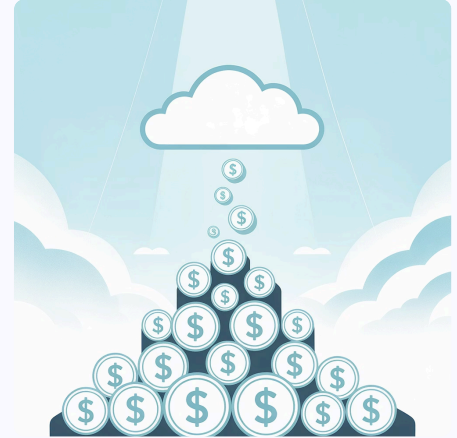


Security and Compliance

Protect models and data, especially in regulated industries.

What to focus on:

- Model security best practices
- Data privacy (GDPR, HIPAA compliance)
- Secure API design
- Model robustness against attacks



Cost Optimization

ML infrastructure can get expensive. Learn to optimize.

What to focus on:

- Choosing right-sized compute resources
- Spot instances and reserved capacity
- Efficient data storage strategies
- Optimizing model inference costs

Who Should Learn MLOps?

MLOps skills can dramatically increase your value and career opportunities. Here are the roles that benefit most:



Data Scientists

Why they need MLOps: Traditionally, data scientists focused on model development. But there's a huge gap between a model in a Jupyter notebook and one running in production. Data scientists who understand how to package models properly, create reproducible training pipelines, and implement versioning are 10x more effective than those who just hand off notebooks to engineering teams.



DevOps Engineers Transitioning to AI/ML Projects

Why they need MLOps: If you're already a DevOps engineer, you have an amazing foundation! But ML systems have unique requirements that traditional DevOps doesn't fully cover - like model drift, experiment tracking, and data versioning.



Cloud Engineers

Why they need MLOps: ML workloads have unique infrastructure needs - GPUs for training, optimized storage for large datasets, specialized networking for distributed training. Cloud engineers who understand these requirements can design cost-effective, scalable ML infrastructure.



Engineering Managers Leading ML Projects

Why they need MLOps: Without MLOps knowledge, managers often underestimate the complexity of deploying ML systems, fail to budget properly for infrastructure, or miss key bottlenecks in the workflow.



Software Engineers Working with ML Teams

Why they need MLOps: As ML becomes part of more products, software engineers increasingly work alongside data scientists. Understanding MLOps helps you integrate ML components into applications effectively.

Conclusion



MLOps bridges the gap between experimental machine learning and production-ready AI systems.

Just as DevOps made it easier to build and release software, MLOps makes it easier to build, test, and deploy machine learning systems that work in the real world.

i Remember: the most important step is to begin. Pick one section of this roadmap and start today. Build something small, share it, get feedback, and keep building. That's how you'll master MLOps.



All the best on your journey! :)



Join 1.3 M+ engineers

Free DevOps and Cloud tutorials



Join 200k+ engineers

Behind-the-scenes and career content